

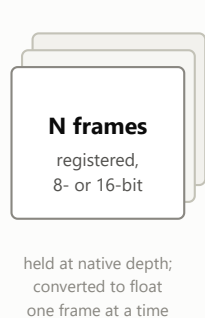
The Depth Map method: how it decides, pixel by pixel

Every other classical method in OpenFocus decides in a transform domain — a pyramid band, a DCT block, a wavelet coefficient. The depth-map family decides **in the image plane**, from a local contrast measure, and it is the only one that names an answer a human can look at: **which frame is sharpest at each pixel**. That map is the method, and both of its selection rules read it.

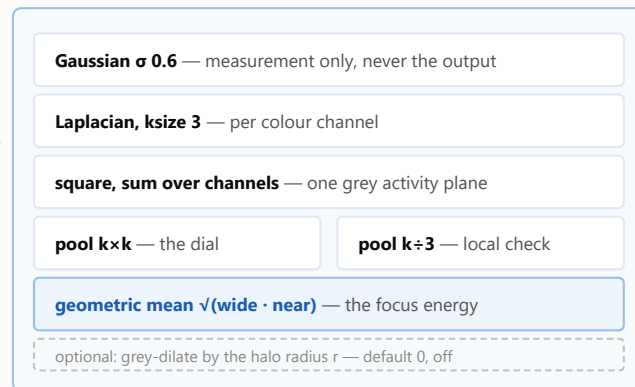
Pages 2–4 are the mechanism as it ships, in the order the pixels move through it. Pages 5–6 set it against **Helicon Focus**, whose Methods A and B are the same two selection rules and whose two dials sit in the same two places — the comparison the method is most often asked to survive.

THE PIPELINE END TO END

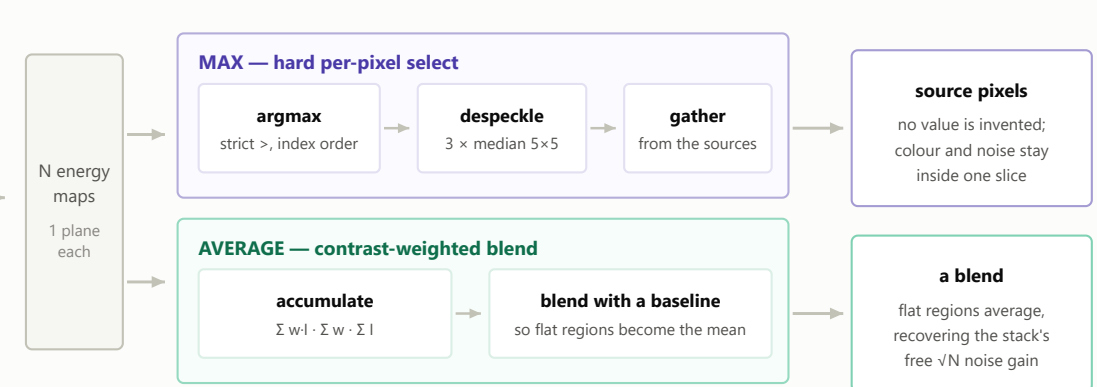
1 · THE STACK



2 · THE MEASURE — once per frame, in parallel



3 · THE DECISION — the only stage that looks at more than one frame



Stages 1–2 never compare frames, so they parallelise across threads (CPU) or batch across a chunk (GPU). Stage 3 is the only reduction, and it is taken in index order — which is what makes the result independent of both scheduling and the order the frames were given in.

■ WHY DECIDE IN THE IMAGE PLANE

A pyramid or wavelet decides on *coefficients*, and a sum of coefficients taken from different frames can land on a value that appears in none of them — the invented pixels the pyramid needs an envelope clamp to prevent. Deciding on pixels cannot do that. The price is that a single window has to stand in for the multiscale prior a transform gets for free, which is what steps 4–6 are about. It is also what makes *Max* auditable: every output pixel traces back to one frame and one coordinate.

■ WHY THE MAP IS THE POINT

Max produces an explicit statement — **frame k is sharpest at this pixel** — which is a range estimate, not just an intermediate. It is what drives 3D, anaglyph and relief output in the tools that expose it, and four of five competitors write it to a file. OpenFocus builds it every render and discards it; see page 6. *Average* produces no map at all — that is the trade it makes for its noise gain.

■ THE TWO DIALS, IN ONE LINE EACH

Kernel — how much neighbourhood a frame has to win over before it is believed. Small follows fine detail and is noisier; large is steadier and reaches further past an edge.

Halo suppression — how far a sharply focused region may claim *beyond* its own outline. Default 0; reach for it only if a glow survives the measure. Not dials: the $\sigma 0.6$ prefilter and the $3 \times 5 \times 5$ despeckle are constants. The second of those is the one control Helicon has and OpenFocus does not — page 6.

2

selection rules over one measure —
hard select (*Max*) and contrast-
weighted average (*Average*)

9 px

default analysis window, adjustable
3–51; one further dial, halo
suppression, defaults to off

0

pixels invented in *Max* mode — every
output pixel is some source pixel, at
the stack's own bit depth

7.9–12.8 $\times$

GPU over CPU, and *Max* is **bitwise
identical** between the two paths

What "sharpest here" is actually measured as

THE CHAIN, PER FRAME

- 1

Smooth it — for measurement only.
`cv2.GaussianBlur(img, (7,7), 0.6)`

A bare 3-tap Laplacian is the most noise-sensitive high-pass there is, and on flat regions the decision was ranking frames by their *noise floor*. The blurred copy is scratch; output pixels are still taken from the untouched frames, so it costs nothing in resolution.
- 2

Take the Laplacian, per channel.
`cv2.Laplacian(img, CV_32F, ksize=3)`

OpenCV's `ksize=3` is the Sobel-derived `[[2,0,2],[0,-8,0],[2,0,2]]` stencil, not the four-neighbour one — the GPU path reproduces those exact taps.
- 3

Square, then sum across channels.
 $E = \sum_c L_c^2$

Summing **before** the decision is what ties a pixel's three channels to one frame. Decide per channel instead and a pixel's red can come from one slice and its blue from another — a colour present in no source. That was DTCWT's bug for years.
- 4

Pool it over the dial's window.
`wide = boxFilter(E, kxk)`

A bare `|∇²|` is zero on flat-but-in-focus areas and noisy everywhere else. Pooling turns it into a *region* measure, so selection follows genuinely sharp regions instead of speckling.
- 5

Pool it again, narrowly.
`near = boxFilter(E, (k÷3)×(k÷3))`

Local evidence that barely leaks past an occlusion — see the figure.
- 6

Combine as a geometric mean.
 $F = \sqrt[\text{max}(\text{wide} \cdot \text{near}, 0)]{}$

A frame has to carry *both* to win. Where the energy really is uniform over the window the two poolings agree and this is exactly the plain pooled energy, so flat regions decide as they always did. The clamp is not cosmetic: `boxFilter` accumulates a running sum and can leave a tiny negative across a sharp edge, and a NaN out of `sqrt` loses every `>` it is compared with.
- 7

Optionally, grey-dilate by the halo radius.
`cv2.dilate(F, ellipse(r)) · r = 0 by default`

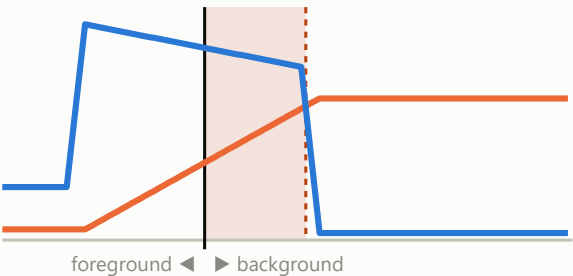
Lets a strongly focused region claim the band its defocused glow contaminates in the other frames. It does not remove a ring so much as fill one with defocused pixels, which is why it is off by default.

WHY TWO WINDOWS AND NOT ONE

Frame holding the foreground contour Frame holding the background texture

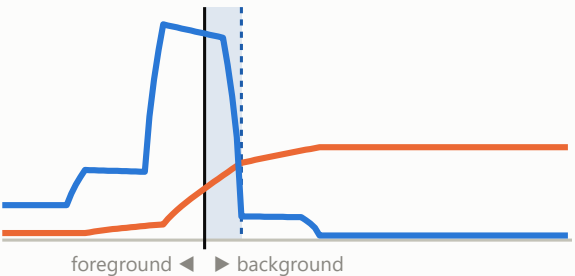
Pooled focus energy, each panel scaled to its own peak · kernel 51, narrow window 17

One pooling window — as it was



► 21 px of background taken from the foreground's frame

Two windows, geometric mean — as shipped



► 7 px of background taken from the foreground's frame

A 1-D slice across an occlusion. A box window is edge-blind, so one pooling smears the contour's enormous energy `51 // 2` px in every direction — including out over background belonging to another slice, which is then taken from a frame where it is defocused. The narrow window barely leaks, so requiring a frame to win on both collapses the band from 21 px to 7.

The narrow window is not always on. It is dropped below 5 px — a window that small hardly leaks, while a 3 px pooling of a squared Laplacian is mostly sensor noise, and letting noise into the decision costs more than the leak it would save. Because it is `k ÷ 3`, that puts the switch-on at **kernel 15**: at the default 9 the measure is the plain pooled energy it has always been, and the correction engages exactly where the ring becomes wide enough to see.

KERNEL	NARROW WINDOW	LEAK BAND IT CAN REACH
9 default	off	4 px
15	5	7 px
31	9	15 px
51	17	25 px

The band is `k // 2`, and `halo_radius` adds to it rather than fighting it: the dilation works on the *already pooled* energy, so `k51 + r30` claims 55 px. That combination is what first reported this.

WHAT THE TWO-SCALE MEASURE WAS WORTH

GROUND-TRUTH SCENARIO, PSNR AT KERNEL 51	ONE WINDOW	TWO	Δ
long_stack	30.77	38.13	+7.36
fine_texture	34.84	39.71	+4.87
low_contrast	60.28	64.07	+3.79
sensor_noise	47.78	50.32	+2.54
deep_stack	27.60	29.49	+1.89
saturated_colour	30.40	31.65	+1.25
depth_edge	34.39	35.45	+1.06

No scenario got worse. On the 333-frame macro stack that reported the ring, detail retained in the band around the subject improved **2.6x**, at a cost of 3.8 s → 4.7 s.

One measure, two ways to spend it

A Max — the hard per-pixel select

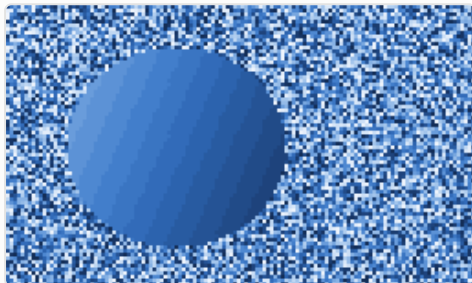
cf. Helicon Method B, Zerene DMap

```
index[y,x] = argmaxk Fk[y,x]    strict >, walked in index order
index      = median5x5( index )    before a single pixel is gathered
out[y,x]   = frame[ index[y,x] ][y,x] at the stack's own depth
```

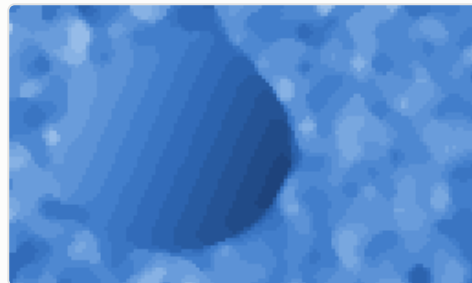
- **The reduction records the winner, not the pixel.** An index map costs one byte per pixel against twelve for a float32 BGR accumulator, and lets the result be gathered at native depth with no float round trip. The map widens to `uint16` past 256 frames and `uint32` past 65 536, so a deep stack cannot wrap.
- **A frame that wins nowhere is never loaded.** The gather walks the frames once and skips any whose index never appears.

WHY THE MAP IS DESPECKLED FIRST

The argmax carries **no spatial prior whatsoever**. Over a region no frame ever resolves — a background behind the focus sweep, a dark flat patch — every frame's energy sits at the same defocused level and the winner is decided by whatever noise survived the pooling. Neighbouring pixels then take frames from opposite ends of the stack, which render that background at visibly different blur and brightness. A depth map is piecewise-smooth: it varies continuously across a surface and steps only at an occlusion, so those pixels are outliers against their own neighbourhood — exactly what a median removes while leaving genuine steps standing.



Straight from the argmax. **69% of background pixels** chose a frame more than 3 slices from their neighbourhood's.



After $3 \times 5 \times 5$ median. **0.2%** — and the subject's own depth moved by 0.05 of a slice.

frame 0 frame 23
far near

Three narrow passes, not one wide median. Iterating a narrow median converges toward its root signal — removing what is smaller than the window without eroding what is larger — where one wide median rounds off real depth features of its own size too. Three 5×5 passes match a single true 9×9 on incoherence while keeping more in-focus detail, and 5 is also the widest window `cv2.medianBlur` takes above `uint8`: the exact 9×9 is a scipy call costing ~1 s on a 1.7 MP map against 15 ms for three cv2 passes. **Measured cost on real stacks:** mean PSNR over seven scenarios rose 41.87 → 42.27 dB at kernel 9 and SSIM 0.969 → 0.982; the one real loss is `Long_stack` at kernel 51, -0.78 dB, and splitting the extra error by row puts **100.5% of it within 4 px of a depth-band boundary**.

B Average — the contrast-weighted blend

cf. Helicon Method A, Shine Stacker average mode

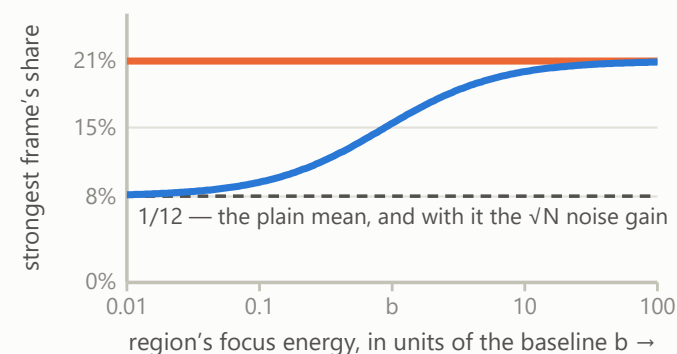
$b = \max(0.1 \cdot \text{mean}(F), 1e-8)$ the baseline weight

$$\text{out} = \frac{\sum_k F_k \cdot I_k + b \cdot \sum_k I_k}{\sum_k F_k + b \cdot N}$$

- **There is no index map**, so there is nothing to despeckle — the blend is coherent by construction, which is the mode's whole argument.
- **The baseline is scale-free.** It tracks a fraction of the stack's own mean energy rather than an absolute level, so it means the same thing on a dark scene as on a bright one. The `1e-8` floor is only so a perfectly flat stack still averages instead of dividing by zero.
- **This is where the $\sqrt{N}$ comes from.** Where every frame is equally defocused the weights flatten and the region collapses to the plain mean of N frames — a free $\sqrt{N}$ noise reduction that every hard-select method throws away by keeping one frame and discarding the rest.

WHAT THE BASELINE ACTUALLY BUYS

With the baseline — as shipped ■ Pure contrast weighting



Twelve frames over a region none of them resolves, where grain alone leaves one frame's energy at $3 \times$ the others'. Pure contrast weighting hands that frame **21.4%** of the output no matter how meaningless the energy is — it is ranking noise. The baseline is a fixed fraction of the *stack's* mean energy, so below it the weights flatten to **8.3%** each and the region becomes the plain mean; above it the sharp frame still takes over.

Which mode to reach for. *Max* keeps colour and noise strictly inside one slice and invents nothing, so it is the honest choice on textured subjects and the only one that produces a depth map at all. *Average* is the answer where the stack is noisy or the subject is smooth — it is the only method in OpenFocus that turns stack depth into signal-to-noise rather than only into depth of field. Neither dominates, which is why the mature tools ship both and teach users to retouch between them.

What the code guarantees that the mathematics does not

■ DETERMINISM & FRAME ORDER

- **The result does not depend on the order frames are given in.** The `argmax` is taken over all frames at once, not folded pairwise, so reversing the stack returns a bit-identical image. This is a real distinction: it is what DTCWT lacked until 1.30.13, and it is what Helicon's own documentation says Method B *cannot* offer.
- **Ties break on the lowest index, always.** Frames are measured on a pool but reduced in index order with a strict `>`, so two renders of the same stack are bitwise equal however the threads happened to be scheduled.
- **MODE_AVERAGE accumulates in index order** for the same reason — float addition is not associative, and `as_completed` ordering would make repeat renders differ by about a level.

■ THE GPU PATH, AND HOW FAR IT IS ALLOWED TO DIFFER

12 FRAMES, 4000×3000, KERNEL 9	CPU	GPU	SPEED-UP
Max	1.12 s	0.14 s	7.9×
Average	2.03 s	0.16 s	12.8×
Max, halo radius 8	1.34 s	0.21 s	6.4×

- **The batch is derived, not pinned.** Frames per chunk come from what `cuda.mem_get_info` reports free against an estimate of a frame's device cost — 15 for a 12 MP stack on a 16 GB card, 6 for 24 MP with halo suppression on — and the chunk halves and retries on `OutOfMemoryError`, so being wrong is slow rather than fatal.
- **The tie-break survives batching.** The measurement is batched but the reduction still walks a chunk one frame at a time, because the strict `>` in index order is what makes both paths pick the same frame. A batched `argmax` would quietly resolve those the other way.
- **The halo element is reproduced, not approximated.** `max_pool2d` gives a square window, which would push the band out to `r*J2` at the corners and make the radius mean something different on each path. The ellipse is decomposed into one horizontal maximum per distinct row half-width, reproducing OpenCV's `round(J(r²-dy²))` spans exactly — asserted bitwise equal to `cv2.dilate` for radii 1, 2, 4 and 8.
- **The median runs on the host,** over a plane that is a couple of megabytes, so both paths go through the same `cv2` call and agree on it exactly; only the pixels it actually moved are re-gathered, and only the chunks holding a frame those pixels now name are re-uploaded.
- Two deliberate departures remain, both at the frame border: `cv2` pools with `BORDER_REFLECT` where torch's reflect padding is `BORDER_REFLECT_101`, and *Average* sums a chunk as a batch rather than frame by frame. Inside the frame the two agree to better than 55 dB; *Max* is bitwise identical.

■ MEMORY, AND WHY IT IS BOUNDED

- **The stack is never held as float.** Frames are normalised on demand, so only the handful in flight cost 12 bytes a pixel; one 60 MP frame is 720 MB as `float32`.
- **The pool is sized by memory, not only by threads.** The count starts at `min(8, cpu_count)` — the OpenCV filters underneath are already internally parallel — then is capped at 25% of the free physical memory divided by an estimate of what one in-flight measurement actually holds. It only ever lowers the count, and says so once when it does.
- **Peak measured** on 12 frames of 4000×3000 at kernel 9: +2.6 GB for *Max*, +2.8 GB for *Average* — down from 3.7 and 3.6 before the index map replaced the float accumulator.

■ BIT DEPTH, END TO END

- **An 8-bit stack returns `uint8`; a 16-bit stack returns `uint16`.** A mixed stack is rescaled once to the common depth and stays there.
- **In *Max* mode the output is a gather, not a cast.** Pixels are copied from the native-depth frames after the decision, so no float round trip stands between a source pixel and the file — every output value is exactly some input value.
- **In *Average* mode** the blend is computed in `float32` and converted once, with rounding rather than truncation.

■ WHAT IS STILL MISSING — READ THIS BEFORE PAGE 5

- **The depth map is computed and discarded.** This method literally builds the artefact four of five competing tools export as a file, and the substrate for 3D and anaglyph output. It cannot be saved.
- **The despeckle is not a dial.** Three passes of 5×5 is a fixed constant, chosen on the fixtures. Helicon exposes precisely this as *Smoothing*, and it is the one parameter where the field has a control OpenFocus does not.
- **There is no contrast or trust threshold.** The `argmax` is taken unconditionally everywhere, including where the measure is pure noise. The median that follows makes the speckle coherent — it does not make the estimate *true*. Zerene marks such regions and stretches the estimate across them instead.
- **No photometric normalisation upstream.** The focus measure is contrast-linear, so a frame 5% brighter wins the `argmax` on identical detail. This is not a defect of this method — it corrupts the input to all of them — but the depth map is where it shows first, as a band of the map jumping to whichever frame the flash favoured.
- **No confidence is surfaced.** `best - second` is available for free at the moment of the decision and is thrown away. "No frame is sharp here" is the most common cause of a disappointing render and is currently invisible.

■ AND WHAT STOPS THE REST OF IT REGRESSING

GUARD	WHAT IT WOULD CATCH
The quality ratchet recorded per method, per scenario	Any drop in PSNR/SSIM against the ground-truth references in <code>tests/fusion_scenarios.py</code> — the seven scenarios on page 2, plus the metric suite in <code>tests/fusion_metrics.py</code> . Re-recorded deliberately when a change is an improvement, never silently.
<code>tests/test_depthmap_halo.py</code>	The halo radius meaning what it says: a wider radius must claim a wider band, and radius 0 must be exactly the undilated measure.
<code>tests/test_depthmap_gpu.py</code>	The whole device path on a <code>cpu</code> torch device — so the batching, the chunk loop and the span-by-span ellipse are exercised on a machine with no card. Includes the bitwise <code>cv2.dilate</code> equivalence and a forward-vs-reversed stack check.
CPU/GPU parity pairs in <code>tests/test_fusion_quality.py</code>	<i>Max</i> bitwise identical at kernels 9, 31 and 51 and for 16-bit frames with halo suppression on; <i>Average</i> within 2 LSB, which is the known float summation-order departure and nothing more.

Sources for this page: `docs/ALGORITHM_IMPROVEMENTS.md` items 20–22 (measured), `docs/ALGORITHM_COMPARISON.md` §4–6.

Against Helicon Focus — the same two rules, different dials

THE METHODS LINE UP ALMOST EXACTLY

SELECTION RULE	OPENFOCUS	HELICON FOCUS 8
Contrast-weighted average blend everything by sharpness	Depth Map (Average) weights + a scale-free baseline, so flat regions become the plain mean	Method A "computes the weight for each pixel based on its contrast and then forms the weighted average of all pixels from all source images"
Hard per-pixel select build a depth map, then gather	Depth Map (Max) order-independent argmax, median-despeckled before gathering	Method B "selects the source image containing the sharpest pixel and uses this information to form the 'depth map'"
Multiscale pyramid not this method — the usual pairing	Pyramid separate method	Method C "uses pyramid approach... increases contrast and glare"

This is the industry's standard pairing and both tools have it: one depth-map family for clean colour and noise, one pyramid for intersecting detail and hair, and neither dominating. Helicon additionally keeps a legacy **Method B1**, documented as faster but producing "more artifacts on uniform backgrounds with brightness variations" — which is, precisely, the failure mode page 3's despeckle exists to remove.

THE DIALS SIT IN THE SAME TWO PLACES

WHAT IT CONTROLS	OPENFOCUS	HELICON FOCUS 8
The analysis neighbourhood	Kernel — the <i>side</i> of the pooling window. Default 9, range 3–51, both modes	Radius — "the number of pixels around each pixel that are used to calculate its contrast". Default 8, Methods A and B only
Smoothing the decision	not exposed fixed at 3 passes of a 5×5 median over the index map (<i>Max</i>); <i>Average</i> needs none	Smoothing — for Method B "determines how the depth map will be smoothed out". Default 4, available in every method
Halos at depth edges	Halo suppression, 0–30 px, independent of the kernel — plus the two-scale measure, which discounts a glow without blurring anything	indirect raise <i>Radius</i> : doing so "allows to almost eliminate the halo effect", though excessive increase "will affect details"
Frame order	irrelevant the argmax is one decision over all frames; reversing the stack is bit-identical	load-bearing Method B "imposes strict requirements on the order of images — it should always be consecutive"; an auto-sort exists to fix it

Coming from Helicon? The two neighbourhood dials are not the same number.

Helicon's *Radius* is described as the pixels **around** each pixel — a radius. OpenFocus's *Kernel* is the **side** of the window. If that reading is right, the like-for-like conversion is $\text{kernel} = 2 \times \text{radius} + 1$:

HELICON <i>RADIUS</i>	3	5	8	12	20
OpenFocus <i>Kernel</i>	7	11	17	25	41

▼ Helicon's default. OpenFocus's own default of 9 sits between Radius 4 and 5. So Helicon's default analyses roughly a **17 px** neighbourhood where OpenFocus's default analyses **9 px** — OpenFocus ships the finer, noisier, more detail-preserving setting of the two. A Helicon user who dials 8 into the OpenFocus kernel slider is asking for a window less than half the size they are used to. This conversion is an inference from the vendor's wording, not a vendor statement — treat it as a starting point and judge by eye.

There is a second, happier consequence: at kernel 17 the two-scale measure is **on** (page 2), so the Helicon-equivalent setting is also the regime where OpenFocus's occlusion correction is doing work.

WHAT EACH TOOL IS ACTUALLY OPTIMISING FOR

Helicon's two dials describe a workflow: **look wider until the halo goes away, then smooth until the seams do**. Both are blunt and both cost detail, and the product's answer to what remains is not a better measure — it is the **retouching brush**, which copies a region from any chosen source frame into the result. The vendor's own documentation treats that as expected practice rather than as a failure.

OpenFocus has no retouching brush, so it cannot make that trade. Its answer to the same two artefacts is instead structural, and made once at the measure: the **two-scale geometric mean** removes the halo by asking a frame to prove the energy is local rather than by widening the window, and the **median on the index map** removes the seams without touching the pixels, so the fix cannot blur anything that was already right. That is why the despeckle is a constant and not a slider — but it is also why not having the slider is a real gap rather than a philosophical one: when the constant is wrong for a stack, there is nothing the user can do about it.

WHAT TO TAKE FROM HELICON, IN ORDER

1. **Expose the despeckle.** Helicon's *Smoothing* and OpenFocus's three median passes do the same job; one is a slider and one is a constant. The mechanism already exists and is shared by both device paths — this is a parameter, a UI row and a range, not an algorithm. It is the smallest item here, and the only place a competitor has a control OpenFocus lacks *inside* the method.
2. **Write the depth map out.** It is built every render and thrown away. Four of five competing tools export it, and it is the substrate for the 3D output Helicon builds on top of it.
3. **Normalise brightness before measuring.** Not this method's bug, but this method's map is where it shows first, and it is the one fix that improves every fusion method at once.
4. **Give the argmax a trust threshold.** The median makes an unresolvable region *coherent*; it does not make it *right*. Marking those regions is what Zerene does, and the confidence signal it needs is already computed and discarded.

Helicon quotations are from *Understanding the Focus Stacking Parameters* and the Helicon Focus 8 user guide (heliconsoft.com), retrieved for this document. Nothing here is derived from Helicon's code, and no claim is made about how its measure is computed — the vendor does not say.

Where each one is ahead, and what to take from the other

CAPABILITY	● OPENFOCUS DEPTH MAP 1.34.0 SOURCE-INSPECTED	● HELICON FOCUS 8, METHODS A/B VENDOR-DOCUMENTED	READING
Focus measure	Squared Laplacian, summed across channels, noise-prefiltered, pooled at two scales and combined as a geometric mean	Not documented; a local contrast measure over the <i>Radius</i> neighbourhood	Not comparable — the vendor does not say. Only OpenFocus's is auditable.
Decision-map smoothing	Fixed: 3 × 5×5 median, not exposed	Smoothing dial, default 4, user-set per render	Helicon the clearest single gap
Halo control	Dedicated radius , independent of the analysis window, <i>plus</i> a measure that discounts a glow on its own merits	Indirect only — widen <i>Radius</i> , losing detail with it	OpenFocus a control the field only offers by side effect
Frame-order independence	Yes — one decision over all frames; reversal is bit-identical	No for Method B — consecutive order is a stated requirement	OpenFocus a correctness property, not a preference
Reproducibility	Bitwise stable run to run, and CPU ≡ GPU in <i>Max</i>	Not documented	OpenFocus claimed by nobody else in the field
Flat-region SNR	Average mode, with a scale-free baseline making the collapse to the mean explicit	Method A , same family	Parity
Depth map as output	discarded computed every render, never written	Saved as a greyscale image; Export 3D Model opens it in Helicon 3D Viewer for conversion to mesh formats	Helicon the method's own artefact, unavailable
Trust / contrast threshold	none unconditional argmax everywhere	Not documented	Neither — Zerene is the tool that has it
Artefact repair after the fact	none	Retouching : copy from source, clone, erase. Dust Map from a defocused white frame	Helicon caps how good any pure-algorithm result can be on hard subjects
Photometric normalisation	none — a 5% brighter frame wins on identical detail	Brightness/exposure matching documented	Helicon corrupts the measure's input, so it outranks method work
Bit depth	Native 8- or 16-bit preserved end to end; in <i>Max</i> every output pixel is a source pixel	"Always processes images with 16-bit precision"	Parity, by different routes
Acceleration	CUDA / MPS, 6.4–12.8×, with the CPU path verified against it	OpenCL, Pro edition; 4 GB GPU recommended for 20–40 MP	Parity
RAW workflow	Nikon <code>.nef</code> / <code>.nrw</code> named, LibRaw underneath	Raw-in / DNG-out (Pro), broad camera support	Helicon outside this method, but it decides what reaches it

■ THE HONEST SUMMARY

On the decision itself, OpenFocus is ahead. Its measure is occlusion-aware where the field's is a single window; its halo control is direct where the field's is a side effect of the analysis radius; its result does not depend on the order frames arrive in, which Helicon's Method B explicitly does; and it is reproducible bit for bit across runs and across devices, which nobody in the comparison claims. Every one of those is measured in `docs/ALGORITHM_IMPROVEMENTS.md` and guarded by tests.

On everything around the decision, Helicon is ahead — and by more than the list above suggests, because the items compound. Helicon's user can smooth the map when the default is wrong, save the map when they want it, retouch what neither method got right, and feed the whole thing properly exposed, dust-free RAW frames. The OpenFocus user gets a better decision and then has to accept it exactly as rendered.

That is the same shape the wider audit found across all eight methods: **more algorithm than anyone, less pipeline than anyone**. For this method specifically the correction is unusually cheap — exposing one constant and saving one array that is already in memory would close the two gaps that sit inside the method's own boundary.

Evidence. OpenFocus entries: source inspection of `fusion_methods/depthmap.py` and `depthmap_torch.py` at 1.34.0, with measurements from `docs/ALGORITHM_IMPROVEMENTS.md` items 20–22 (synthetic ground-truth fixtures in `tests/fusion_scenarios.py` plus one 333-frame real macro stack, on one machine). Helicon entries: *Understanding the Focus Stacking Parameters* and the Helicon Focus 8 user guide. Helicon Focus is closed source; its column describes **claimed** behaviour and no head-to-head render was made.